Miscellaneous Interview Questions

SQL/MongoDB, system design, DevOps & cloud, CSS/HTML, OOP, and behavioral/resume questions

Compiled from real interview experiences (The future University, CREATE Bytes, Innovation Labs, Seventh Triangle, qpiAI) + curated additions

---

Topics covered: SQL joins & normalization · MongoDB aggregation/replication/sharding · SOLID/KISS/DRY · system design scenarios · Docker/Nginx/AWS/Azure · CSS positioning & preprocessors · OOP fundamentals · behavioral & resume-based questions
Difficulty split: Easy → Medium → Hard, spanning theory, scenario-based, and design-discussion questions

Format: Every section is split into **Easy** → **Medium** → **Hard**, and spans theoretical concepts, output-based/tricky code, and practical / machine-coding style problems.

EASY LEVEL

HTML & CSS

1. What is the difference between inline, block, and inline-block elements? Give one HTML tag example of each.

2. What are CSS preprocessors (SASS/SCSS)? Name 2 features they offer that plain CSS didn't have until recently (variables, nesting, mixins).

3. Explain the CSS position property values: static, relative, absolute, fixed, and sticky — with the key difference between each.

4. How would you implement a simple CSS animation/transition (e.g. an element sliding left to right on hover)?

OOP Basics

5. What are the 4 pillars of Object-Oriented Programming? Explain each with a one-line example.

6. What is the difference between method overloading and method overriding?

7. What is the difference between an abstract class and an interface?

8. What is encapsulation and why is it useful in large codebases?

SQL Basics

9. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

10. What is normalization? Briefly explain 1NF, 2NF, and 3NF.

11. What is a primary key vs a foreign key vs a unique key?

12. Write a query to find the top 10 maximum-paying customers from a customers/orders schema.

General DevOps Basics

13. What is Docker and why is containerization useful compared to running an app directly on a VM?

14. What is Nginx typically used for in a production web stack (reverse proxy, load balancer, static file serving)?

15. What is the difference between a load balancer and a reverse proxy? (They can overlap — explain how.)

MEDIUM LEVEL

Database Design & Relations

1. Explain 1:1, 1:N, and N:N relationships with practical schema examples in both SQL and MongoDB (embedding vs referencing).

2. Write a SQL query to find all artists who have made any song in the 'Comedy' genre (multi-table join).

3. What is database indexing? How does an index speed up reads, and what's the trade-off on writes?

4. What is a covered query in MongoDB and why is it faster?

■ *A query where all fields are part of an index, so MongoDB can return results using only the index without fetching the full document*

5. Explain Mongoose virtuals — what are they used for, and how are they different from a stored field?

6. Which MongoDB operator is used to specify a range of values in a query, and how does it differ from SQL's BETWEEN?

System Design Concepts

7. Explain SOLID principles briefly, one line per principle, with a code-smell example each.

8. Explain the KISS and DRY principles and why over-engineering is a common anti-pattern in early-stage projects.

9. Explain common design patterns used in backend systems: the Factory pattern and the Modular pattern, with a use case each.

10. What is the difference between horizontal and vertical scaling? What are the limits of vertical scaling?

11. Explain caching strategies (cache-aside, write-through, write-back) and where Redis typically fits into a web architecture.

Networking / Protocols

12. Compare HTTP vs HTTPS vs TCP vs UDP — which protocol does WebSocket use underneath and why is that important for real-time apps?

13. Explain HTTP headers in detail — what are Content-Type options like text/plain, application/json, and multipart/form-data used for, and when does each apply?

14. When a form is submitted with an image, in what format is the image typically sent to the backend, and how do you handle that on the server?

15. Read about WebSockets — what are their advantages and disadvantages compared to traditional polling?

DevOps / Infra

16. Explain the basics of a Docker + Nginx setup for deploying a Node.js/React app — Dockerfile, reverse proxy config, and SSL termination.

17. What is the AWS Incremental Migration / DataSync approach for syncing large datasets to the cloud, and when would you use it over a one-time bulk transfer?

18. Explain the difference between Azure RBAC and Entra ID (Azure AD) directory roles.

19. What is IAM in AWS, and how would you design a least-privilege S3 read/write policy for a specific folder prefix?

Behavioral / Resume-based

20. Walk me through your resume in detail — technical skills, experience, and projects, explaining the 'why' and 'how' behind each choice.

21. Where do you see yourself in the next 5 years?

22. Describe a project you're most proud of and the specific technical challenges you solved in it.

23. How do you decide which project to showcase in an interview? What should you double-check before presenting it?

HARD LEVEL

Advanced Database / MongoDB

1. Explain MongoDB replication and replica sets in depth — how does failover work, and what is the role of the oplog?

2. Explain MongoDB sharding — how is data distributed across shards, what is a shard key, and how do you choose a good one?

3. Design a MongoDB aggregation pipeline that uses $lookup (relations/populate equivalent), $facet (parallel pipelines), $group, and $unwind together to produce a paginated report with a total count in a single query.

4. Which of the following correctly ensures strong consistency in a distributed MongoDB environment, and why: write concern {w:1} + read preference primaryPreferred, OR write concern {w:'majority'} + read preference primary?

■ *{w:'majority'} + read preference primary — ensures the write is acknowledged by a majority of replicas before being considered successful, and reads always hit the primary for the latest data*

5. Design a normalized SQL schema with proper JOINs for a multi-tenant SaaS product (users, orgs, roles, permissions) and explain your normalization decisions.

System Design — Scenario Based

6. Design a reliable multi-step image processing pipeline (upload → preprocess → external API calls → final storage & delivery) that processes tens of thousands of images per day with potential failure at every step. How do you guarantee every image progresses as far as possible (idempotency, retries, dead-letter queues, checkpointing)?

7. Design a system to automatically group real estate photos into 'brackets' (same scene shot with different exposures in quick succession) out of hundreds of uploaded photos per shoot.

8. Design a scalable REST API that can handle a very high read/write ratio — discuss caching, read replicas, CDN, and rate limiting together as a cohesive strategy.

9. Design horizontal scaling for a WebSocket-based real-time chat application across multiple server instances (sticky sessions vs Redis pub/sub adapter).

10. Design a webhook delivery system that guarantees at-least-once delivery to third-party endpoints, including retries with exponential backoff and signature verification.

11. Design a rate limiter used across multiple distributed API server instances (not just a single in-memory limiter) — discuss Redis-based token bucket / sliding window implementations.

Cloud / DevOps — Advanced

12. Design an end-to-end migration plan for moving a production app + database (e.g. Azure to AWS) with minimal downtime — inventory, data sync strategy (e.g. DataSync/DMS), cutover plan, and rollback plan.

13. Compare Amazon Aurora vs RDS MySQL for a single-replica, cost-sensitive production workload — what factors would push you toward one or the other?

14. Explain a zero-downtime blue-green or rolling deployment strategy for a containerized Node.js app behind Nginx/a load balancer.

15. Design an S3 security architecture for user-uploaded content: presigned URLs generated by the backend, Block Public Access, and folder-level IAM scoping.

OOP / Software Engineering Principles

16. Explain composition over inheritance with a practical example of where inheritance creates a fragile hierarchy.

17. Walk through the basic principles of software engineering applied to a new project: requirements gathering → tech stack selection → architecture design. What trade-offs do you weigh at each phase?

18. Explain common OOP interview questions in depth: polymorphism (compile-time vs runtime), abstraction vs encapsulation, and the diamond problem in multiple inheritance.

Aptitude / Puzzle-style

19. Classic bottle/poison puzzle: You have N bottles and one is poisoned. Using the minimum number of test subjects/strips, how do you identify the poisoned bottle? Explain the binary-encoding logic behind the optimal solution.

20. You're asked to estimate how many API requests a system needs to handle per second given X daily active users with Y average actions per session — walk through a back-of-the-envelope estimation.